

TypeScript Problem-Solving Assignment

General Rules

- **Do not change the function names.** Use the exact name given in each problem.
- Every problem must be solved using a **function**.
- **Every function must return a value.** Do not use `console.log()` as the final answer.
- Use appropriate TypeScript types for function parameters, function return values, objects, and arrays.
- Avoid using `any`.
- You may use JavaScript/ES6 features such as `map()`, `filter()`, `reduce()`, and template literals.
- Check your function against every row in the test case table, not just the first one.
- Think about edge cases — like an empty array — before considering a problem complete.

Problem 1 — Battery Level Status

Function name must be: `getBatteryStatus`

Scenario

A phone's settings screen shows a friendly battery status label instead of just the raw percentage.

Task

Create a function named `getBatteryStatus`. The function receives the battery percentage (0–100) and returns a status string.

Status Rules

Percentage	Status
0–20	"Low"
21–50	"Medium"
51–90	"High"
91–100	"Full"

Function Requirement

- Accept the percentage as a `number`.
- Return the status as a `string`.

Starter Code

```
function getBatteryStatus(percentage: <type>): <type> {  
    // write your code here  
}
```

Test Cases

Input	Output
10	"Low"
35	"Medium"

Input	Output
75	"High"
100	"Full"

Problem 2 — Table Booking Confirmation

Function name must be: **formatBookingConfirmation**

Scenario

A restaurant booking app confirms a table reservation with a short readable message. A booking contains `name`, `guests`, and `time`.

Task

First define an appropriate type or interface for the booking. Then create a function named `formatBookingConfirmation` that receives a booking and returns a confirmation sentence.

Requirements

- Accept a properly typed booking object.
- Return a `string`.
- Use the values from the object rather than hard-coding the result.

Starter Code

```
function formatBookingConfirmation(booking: <type>): <type> {  
    // write your code here  
}
```

Test Cases

Input	Output
<code>{ name: "Aisha", guests: 4, time: "7:00 PM" }</code>	"Aisha's table for 4 guests is confirmed at 7:00 PM."
<code>{ name: "Rahim", guests: 2, time: "8:30 PM" }</code>	"Rahim's table for 2 guests is confirmed at 8:30 PM."

Problem 3 — Weekly Expense Tracker

Function name must be: **calculateWeeklyTotal**

Scenario

A budgeting app lets a user log each expense for the week as a plain number, and needs to add them all up.

Task

Create a function named **calculateWeeklyTotal**. The function should receive an array of expense amounts and return the total.

Requirements

- Properly type the array of numbers.
- Return the total as a **number**.
- An empty array should return **0**.

Starter Code

```
function calculateWeeklyTotal(expenses: <type>): <type> {  
    // write your code here  
}
```

Test Cases

Input	Output
[200, 450, 100]	750
[1000, 250]	1250
[]	0

Problem 4 — Traffic Light Action

Function name must be: `getTrafficAction`

Scenario

A driving simulator app tells the player what action to take based on the current traffic light color.

Task

First create a **union type** `Light`, then create a function named `getTrafficAction` that receives a `Light` and returns the correct action.

Rules

Light	Action
red	"Stop"
yellow	"Slow Down"
green	"Go"

Starter Code

```
function getTrafficAction(light: <type>): <type> {  
  // write your code here  
}
```

Test Cases

Input	Output
"red"	"Stop"
"yellow"	"Slow Down"
"green"	"Go"

Problem 5 — Quiz Score Summary

Function name must be: **getQuizSummary**

Scenario

A quiz app wants to show a simple summary of a student's scores across all questions: the total and the average.

Task

Create a function named `getQuizSummary`. The function should receive an array of scores and return an object containing `total` and `average`.

Starter Code

```
function getQuizSummary(scores: <type>): <type> {  
    // write your code here  
}
```

Test Cases

Input	Output
[8, 9, 7, 10]	{ total: 34, average: 8.5 }
[5, 5]	{ total: 10, average: 5 }
[]	{ total: 0, average: 0 }

